Software Engineering 考试复习 (完全按照你的 PPT)

> 老大，这份资料 100% 来自你发的 10 个 lecture slides，没有掺别的来源。**答题内容是英文（考试直接用），讲解是中文。**考试 2 小时 / 4 题 / 每题 25 分。下面按你给的结构逐题准备。

我准备了一个**贯穿例子**（Q1 和 Q3 都用它示范，因为这两题都是 "based on scenario"）。考试给你的 scenario 会不一样，但**方法和定义一模一样**，你照搬就行。

📌 贯穿例子场景 (Sample Scenario)

System Title: Online Hotel Booking System (OHBS)

> The Online Hotel Booking System is a web-based application that allows customers to search for available hotel rooms, view room details and prices, make a booking, and pay online. Customers must register and log in before booking. After a successful payment, the system sends a booking confirmation to the customer. Hotel administrators can add, update, and delete room information, set room prices, and approve or reject booking requests. The system must keep customer data secure and respond quickly even when many users access the system at the same time.

考试时你拿到 scenario，先**圈出动作词**（search, login, book, pay, add/update/delete, approve/reject, send）→ 这些是 functional；**圈出形容词/质量词**（secure, fast, web-based）→ 这些是 non-functional。下面教你怎么用。

🟦 Question 1 (25 marks) — Requirements Engineering

来源：Lecture 4 Part 1 & Part 2

Q1a — 定义 + 从 scenario 找出 4 种 requirement

题目要你对给定的系统，**针对 scenario** 做 4 件事（这是题目要求的答题套路，每种 requirement 都要按这个写）：

1. **No define** → 先写定义 (definition)
2. **What is the requirement** → 这个 requirement 是什么
3. **Depend on the scenario given** → 必须基于给的 scenario
4. **Answer inside the scenario** → 答案要能在 scenario 里找到根据

下面是 4 种 requirement 的**标准定义（背这个）+ 用 OHBS 示范。**

---

① Functional Requirement (功能需求 — What the system *does*)

Definition (PPT 原话):

> Functional requirements are statements of services the system should provide, how the system should react to particular inputs, and how the system should behave in particular situations. They may also state what the system should *not* do.

怎么找： scenario 里的**动作/功能**就是 functional。

Answer based on the scenario (OHBS):

- The system must allow customers to search for available hotel rooms.
- The system must allow customers to register and log in before booking.
- The system must allow customers to make a booking and pay online.
- The system must allow administrators to add, update, and delete room information.
- The system must allow administrators to approve or reject booking requests.
- The system must send a booking confirmation to the customer after successful payment.

---

② Non-Functional Requirement (非功能需求 — How *well* the system works)

Definition (PPT 原话):

> Non-functional requirements are constraints on the services or functions offered by the system, such as timing constraints, constraints on the development process, and standards. They often apply to the system as a whole rather than individual features or services.

怎么找： scenario 里讲**质量、速度、安全、可用性**的就是 non-functional。

Answer based on the scenario (OHBS):

- **Performance:** The system must respond quickly even when many users access it at the same time.
- **Security:** The system must keep customer data secure (e.g. encrypted passwords).
- **Availability/Accessibility:** The system is web-based and must be accessible via web browsers.
- **Usability:** The booking interface must be user-friendly and easy to navigate.

> 💡 PPT 把 non-functional 再分成 3 类（如果题目问到可以加）：
> - **Product requirements** — 系统本身要怎样表现，例如 "load each page within 2 seconds"。
> - **Organisational requirements** — 公司规定/标准，例如 "must be developed using C# and ASP.NET"。
> - **External requirements** — 外部因素/法律，例如 "must comply with PDPA Malaysia"。
> - （还有 **Usability requirements** — 易用性。）

---

③ User Requirement (用户需求)

Definition (PPT 原话):

> User requirements are high-level statements that describe the services and functions the system should provide. They are written from the user's perspective in simple and understandable, non-technical language, and are easy for users/stakeholders to understand.

关键区别： user requirement = **给用户看的、高层、非技术语言。**

Answer based on the scenario (OHBS):

- The system should allow customers to search for and book hotel rooms online.
- The system should allow customers to pay for their booking online.
- The system should allow the administrator to manage room information.
- Customers should receive a confirmation after a successful booking.

---

④ System Requirement (系统需求)

Definition (PPT 原话):

> System requirements are detailed and precise descriptions of the system's functions, features, and constraints, written in a technical manner for developers. They are more detailed and specific, written in technical language, and used by developers for implementation.

关键区别： system requirement = **给开发者看的、技术细节**（讲 database、table、authentication 这些）。

Answer based on the scenario (OHBS):

- The system shall authenticate users using a username and password stored in the database.
- The system shall provide a CRUD interface (Create, Read, Update, Delete) for room records.
- The system shall store booking details in a Booking table (BookingID, RoomID, CustomerID, Date).
- The system shall redirect valid admin users to the admin dashboard page.

⚠️ User vs System / Functional vs Non-functional 一句话记牢

对比	一句话区分
User vs System	User = 用户能懂的高层语言；System = 给开发者的技术细节（database / table / authentication）
Functional vs Non-functional	Functional = 系统**做什么**（动作）；Non-functional = 系统**做得多好**（速度/安全/易用）

考试如果 4 种都要写，记得**每一种都重复那 4 步**：先定义 → 再说是什么 → 基于 scenario → 答案在 scenario 里找得到。

Q1b — Data Collection Method: Interview (访谈)

来源：Lecture 4 Part 2

Definition:

> Formal or informal interviews with stakeholders are part of most Requirements Engineering processes. Interviewing is a requirements discovery technique used to gather information, understand requirements, and identify problems.

Two types of interview (一定要写这两种 + 例子):

1. Closed interview — based on a pre-determined list of questions.

- Example (OHBS): *"Does the current system support online payment?"* / *"Is booking confirmation sent by email?"*

2. Open interview — various issues are explored with stakeholders (open-ended).

- Example (OHBS): *"What challenges do users face when booking a room?"* / *"Which part of the system needs the most improvement?"*

Interviews in practice (加分点):

- Normally a **mix** of closed and open-ended interviewing.
- ✅ **Good for:** getting an overall understanding of what stakeholders do and how they might interact with the system.
- ❌ **Not good for:** understanding domain requirements — requirements engineers may not understand the technical terms used in the business domain (e.g. a hospital staff uses medical terms the developer does not understand).

> 💡 如果想多写几行，PPT 还讲了其他 requirements discovery 方法可以顺带提：**Scenarios**（一步步描述用户怎么用系统）、**Use cases**（谁是 actor + 想做什么 + 系统怎么回应）、**Ethnography**（在真实环境观察用户）。但题目指定 interview，主答 interview。

🟩 Question 2 (25 marks) — Architecture

来源：Lecture 6 Architectural Design

> 开头可以先写一句通用定义（任何架构题都能用）：*Architectural Design is the process of defining the overall structure of a software system. It identifies the components/modules, their relationships, communication between them, and the technology used. It acts as a blueprint for software development.*

Q2a (10 marks) — Layered Architecture + Client-Server Architecture

① Layered Architecture (3-Tier Architecture)

Definition:

> Layered architecture divides the system into multiple layers where each layer performs a specific function.

The 3 layers/tiers:

Layer / Tier	作用	技术 (PPT)
Presentation Tier (Client / Front end)	Top level tier. User Interface (UI). Collect and display information.	HTML, CSS, Bootstrap (GUI)
Application Tier (Logic / Middle tier)	Information is processed. Business logic and rules. Add, delete, or update data.	Python, Java, C#
Data Tier (Back end / Database server)	Data are stored and managed.	MySQL, SQL Server

Advantages: Easy maintenance, better organization, easier testing, improved security.
Disadvantages: Slower performance due to multiple layers; more complex for small systems.

---

② Client-Server Architecture

Definition:

> Client-Server is an architecture where clients send requests and servers process and respond.

Components (PPT):

- **Client** — the user side (web browser, mobile app). Functions: register, login, browse, place order, make payment.
- **Web Server** — receives requests from clients, manages communication, serves webpages, forwards requests to the application server.
- **Application Server** — processes the business logic.
- **Database Server** — stores all system information.

Advantages: Centralized management, better security, supports multiple users.
Disadvantages: Server failure affects all clients; network dependency.

> ✍️ 10 分题：两种架构各写「定义 + 层/组件 + 优点 + 缺点」就拿满分。

Q2b (15 marks) — MVC Architecture + 画图 ⭐

Model-View-Controller (MVC)

Definition:

> MVC separates the system into three parts — Model (handles data), View (user interface), and Controller (processes requests).

三个部分详细 (PPT):

Model — manages data, business rules, and database operations. It communicates directly with the database.

- *Example:* When a customer searches for a product, the Model retrieves the product data from the database.

View — the user interface displayed to customers and administrators. Displays webpages, product lists, shopping cart, payment forms, dashboards.

- *Example:* The customer sees the product name, price, and "add to cart" button — this is displayed by the View.

Controller — acts as the **middle layer between View and Model.** It processes user requests and controls system flow. Functions: receive login request, process order, validate payment, call Model, update View.

- *Example:* Customer clicks "add to cart" → Controller receives the request → calls the product Model → updates the cart → returns the updated View.

Advantages: Easier maintenance, reusable components. **Disadvantages:** More files and complexity, requires proper planning.

🖊️ MVC 图 (照着画 — 来自 PPT slide 18)

```
      ┌───────────────┐            ┌───────────────┐
      │     MODEL     │            │     VIEW      │
      │ Handles data  │            │ Handles data  │
      │ logic.        │            │ presentation. │
      │ Interacts with│            │ Dynamically   │
      │ database.     │            │ rendered.     │
      └───────┬───────┘            └───────┬───────┘
              ▲                            ▲
              │ 2. Get Data                │ 3. Get Presentation
              ▼                            ▼
 ┌────┐   ┌──────────────────────────────────────┐
 │    │   │              CONTROLLER              │
 │ 💻 │──▶│ Handles request flow.                │
 │User│   │ Never handles data logic.            │
 │    │◀──│                                      │
 └────┘   └──────────────────────────────────────┘
   1. Request ──▶          ◀── 4. Response
```

画图口诀（4 个箭头方向一定要标对）：

1. **User → Controller**：`1. Request`
2. **Controller ↔ Model**：`2. Get Data`（双向）
3. **Controller ↔ View**：`3. Get Presentation`（双向）
4. **Controller → User**：`4. Response`

> 关键点（PPT 强调）：**Controller 处理流程，但永远不碰 data logic**（"Never handles data logic"）。Model 才管 data。考试画完图记得写这句。

🟨 Question 3 (25 marks) — Software Testing

来源：Lecture 7 Software Testing

Q3a (5 marks) — Verification vs Validation (based on scenario)

Verification:

> Verification is the process of checking whether the software is being **developed according to the specified requirements**. *Example (OHBS):* The team reviews the requirements document and design diagrams to ensure they are correct **before coding begins**.

Validation:

> Validation is the process of checking whether the developed software **meets the user's needs and expectations**. *Example (OHBS):* The team tests the OHBS with real customers to ensure it works as expected and satisfies their needs.

一句话记区分：

- **Verification = "Are we building the product *right*?"** (照不照需求做 / 还没跑程序，靠 review)
- **Validation = "Are we building the *right* product?"** (用户满不满意 / 真的跑程序、给用户测)

> 加分：可以提 Verification 常用 **inspection**（不跑程序、看文件/code 找错），Validation 常用 **testing**（跑程序）。

Q3b (12 marks) — 4 种 Testing（定义 + 基于 scenario 应用）

题目：基于 scenario，你能做哪些测试。**每种写：定义 + 在 OHBS 里怎么做。**

① Unit Testing

Definition: Unit testing is the process of testing individual components/units **in isolation**. It is a **defect testing** process. Units may be individual functions/methods, object classes, or composite components. **OHBS example:** Test the `CalculateTotalPrice()` method alone to check it returns the correct room price.

② Integration Testing

Definition: Integration testing is the process of testing **two or more modules together** to ensure they interact and communicate correctly. It is performed **after Unit Testing and before System Testing**. Purpose: verify integrated components work together, detect defects in interfaces and data flow. **OHBS example:** Test the **Login module + Booking module** together — after login, the customer ID must be passed correctly to the Booking module so the right customer's booking is created.

③ System Testing

Definition: System testing is the process of testing the **complete and integrated software system as a whole** to verify that it meets the specified requirements and functions correctly. Purpose: verify the entire system works, all modules integrated properly, check functional **and** non-functional requirements. **OHBS example:** Test the full booking flow end-to-end — search room → login → book → pay → receive confirmation — to confirm the whole OHBS works.

④ User Acceptance Testing (UAT)

Definition: Acceptance testing is where **customers test a system to decide whether or not it is ready to be accepted** from the developers and deployed in the customer environment. Primarily for custom systems. **OHBS example:** The hotel staff test the OHBS in their own environment and decide whether to accept it.

UAT 的 6 个 stages（如果要多写）：

1. Define acceptance criteria → 2. Plan acceptance testing → 3. Derive acceptance tests → 4. Run acceptance tests → 5. Negotiate test results → 6. Reject/accept system

---

Q3c (8 marks) — Create Test Cases ⭐

来源：Lecture 7 test case 表格格式。**严格用这个格式（PPT 原表头）：**

> 题目会给你一个 requirement，你照下面表格做 1-3 个 test case。

Test Case Template (照抄表头)

Field	内容
Test Case ID	TC001
Module	(模块名)
Test Case Description	(测试什么)
Preconditions	(前置条件)
Test Steps	(1, 2, 3… 步骤)
Test Data	(测试数据)
Expected Result	(预期结果)
Actual Result	(实际结果)
Status	Pass / Fail
Remarks	(备注)

✅ 范例 1 — 基于需求「Customer can log in with valid credentials」

Field	Value
Test Case ID	TC001
Module	User Login
Test Case Description	Verify that a customer can log in with valid credentials
Preconditions	Customer must be registered with valid credentials
Test Steps	1. Go to login page 2. Enter valid email 3. Enter valid password 4. Click "Login" button
Test Data	Email: user@example.com / Password: Test@123
Expected Result	Customer is redirected to the dashboard
Actual Result	Customer successfully logged in and redirected to dashboard
Status	Pass
Remarks	Login feature works as expected with valid inputs

✅ 范例 2 — 基于需求「Customer can book a room」

Field	Value
Test Case ID	TC002
Module	Room Booking
Test Case Description	Verify that a customer can successfully book a room with valid details
Preconditions	Customer must be logged in and rooms must be available
Test Steps	1. Go to Room Booking page 2. Select available room 3. Choose booking date and time 4. Enter booking details 5. Click "Book Room" button
Test Data	Room: Deluxe Room A / Date: 20/06/2026 / Time: 2:00 PM
Expected Result	Room booking is successfully saved and a confirmation message is displayed
Actual Result	Customer successfully booked the room and confirmation message displayed
Status	Pass
Remarks	Booking module functions correctly with valid inputs

> 💡 8 分题：做 2 个 test case（一个 valid、一个可以做 invalid 测 defect），每个表格写满 10 行就够。想测 defect 的话：例如 login 留空 username → Expected Result: 系统显示 "Username is required" 错误信息。

🟥 Question 4 (25 marks) — Design Patterns

来源：Lecture 8 Software Quality Assurance & Design Patterns

> 通用开头（任何 pattern 题都能写）：*A Design Pattern is a reusable and proven solution to a common problem in software design. It is a template (not a complete implementation) that can be adapted to different situations, representing best practices developed by experienced software engineers.*

5 个 pattern 分类（先记分类）：

Category	Patterns
Structural Patterns	Adapter, Decorator
Behavioral Patterns	Observer, Strategy
Dependency Management Pattern	Dependency Injection (DI)

Q4a (15 marks) — 选一个 pattern：是什么 + 怎么运作

下面 5 个全部准备好（题目会让你选一个）。**每个都给：定义 + 目的 + 优点 + 怎么运作 + PPT 的 C# 代码。**

---

① Adapter Pattern (Structural)

What it is: The Adapter Pattern allows **incompatible interfaces to work together.** It acts as a **bridge** between an existing (legacy) system and a new system **without modifying either system. Purpose:** Integrate old systems with new systems. **Advantages:** Reuse legacy system, reduce code modifications, improve compatibility.

How it works (用 PPT 的例子讲): 旧系统有 `ProcessPayment()`，新系统要的是 `MakePayment()`。不改旧 code，建一个 **PaymentAdapter** 实现新接口 `IPayment`，在里面调用旧的 `ProcessPayment()`。

PPT Code:

```csharp
// Old System (Legacy)
public class LegacyPaymentGateway
{
    public string ProcessPayment(decimal amount)
    {
        return "RM " + amount + " paid using Legacy Payment Gateway.";
    }
}

// New System expects this interface
public interface IPayment
{
    string MakePayment(decimal amount);
}

// The Adapter
public class PaymentAdapter : IPayment
{
    private LegacyPaymentGateway legacyGateway;

    public PaymentAdapter()
    {
        legacyGateway = new LegacyPaymentGateway();
    }

    public string MakePayment(decimal amount)
    {
        return legacyGateway.ProcessPayment(amount);  // bridge to old method
    }
}
```

---

② Decorator Pattern (Structural)

What it is: The Decorator Pattern **adds functionality dynamically without modifying existing code.** It wraps an existing object and adds extra functionality while keeping the original object's structure unchanged. **Purpose:** Extend behavior at runtime. **Advantages:** Flexible extension, follows the Open/Closed principle, no modification of the original class.

How it works (PPT 例子): 基础 `StudentReport` 只显示名字。要加 CGPA、attendance，不用建 `StudentReportWithCGPA`、`StudentReportWithAttendance` 一堆类（class explosion）。而是用 decorator 一层层包：先包 `CGPADecorator`，再包 `AttendanceDecorator`。

PPT Code:

```csharp
public interface IReport
{
    string GenerateReport();
}

public class StudentReport : IReport
{
    public string GenerateReport()
    {
        return "Student Name: John";
    }
}

public class CGPADecorator : IReport
{
    private IReport report;
    public CGPADecorator(IReport report) { this.report = report; }
    public string GenerateReport()
    {
        return report.GenerateReport() + "\nCGPA: 3.75";
    }
}

public class AttendanceDecorator : IReport
{
    private IReport report;
    public AttendanceDecorator(IReport report) { this.report = report; }
    public string GenerateReport()
    {
        return report.GenerateReport() + "\nAttendance: 90%";
    }
}

// Client: wrap layer by layer
IReport report = new StudentReport();
report = new CGPADecorator(report);
report = new AttendanceDecorator(report);
Console.WriteLine(report.GenerateReport());
```

---

③ Observer Pattern (Behavioral)

What it is: The Observer Pattern establishes a **one-to-many relationship** where observers automatically receive updates. When one object (the **Subject**) changes its state, all dependent objects (the **Observers**) are automatically notified and updated. **Purpose:** Implement event-driven systems.

How it works (PPT 例子): INTI 公告系统：admin 发公告（Subject），所有订阅的学生（Observers）自动收到通知。

PPT Code:

```csharp
// Step 1: Observer interface
public interface IObserver
{
    void Update(string message);
}

// Step 2: Subject class
public class Announcement
{
    private List<IObserver> observers = new List<IObserver>();
    public void Subscribe(IObserver observer)   { observers.Add(observer); }
    public void Unsubscribe(IObserver observer) { observers.Remove(observer); }
    public void PostAnnouncement(string message) { NotifyObservers(message); }
    private void NotifyObservers(string message)
    {
        foreach (var observer in observers)
            observer.Update(message);
    }
}

// Step 3: Observer class
public class Student : IObserver
{
    private string name;
    public Student(string name) { this.name = name; }
    public void Update(string message)
    {
        Console.WriteLine(name + " received: " + message);
    }
}

// Step 4: Client
Announcement announcement = new Announcement();
Student s1 = new Student("John");
Student s2 = new Student("Mary");
announcement.Subscribe(s1);
announcement.Subscribe(s2);
announcement.PostAnnouncement("Class cancelled tomorrow.");
```

---

④ Strategy Pattern (Behavioral)

What it is: The Strategy Pattern **encapsulates interchangeable algorithms.** It allows you to choose different ways of performing a task without changing the code that uses it. **Purpose:** Switch algorithms dynamically.

How it works (PPT 例子): 计算折扣：No Discount / Student Discount (10%) / Member Discount (20%)。每种折扣是一个 strategy 类，`ShoppingCart` 接收一个 strategy，运行时换算法不用改 `ShoppingCart`。

PPT Code:

```csharp
// Step 1: Strategy interface
public interface IDiscountStrategy
{
    double CalculateDiscount(double amount);
}

// Step 2: Concrete strategies
public class NoDiscount : IDiscountStrategy
{
    public double CalculateDiscount(double amount) { return amount; }
}
public class StudentDiscount : IDiscountStrategy
{
    public double CalculateDiscount(double amount) { return amount * 0.90; }
}
public class MemberDiscount : IDiscountStrategy
{
    public double CalculateDiscount(double amount) { return amount * 0.80; }
}

// Step 3: Context class
public class ShoppingCart
{
    private IDiscountStrategy discountStrategy;
    public ShoppingCart(IDiscountStrategy discountStrategy)
    {
        this.discountStrategy = discountStrategy;
    }
    public double GetFinalPrice(double amount)
    {
        return discountStrategy.CalculateDiscount(amount);
    }
}

// Step 4: Client
double amount = 100;
ShoppingCart cart1 = new ShoppingCart(new StudentDiscount());
Console.WriteLine("Student Price: RM " + cart1.GetFinalPrice(amount));
ShoppingCart cart2 = new ShoppingCart(new MemberDiscount());
Console.WriteLine("Member Price: RM " + cart2.GetFinalPrice(amount));
```

---

⑤ Dependency Injection (DI) (Dependency Management)

What it is: Dependency Injection is a design pattern where **dependencies are supplied externally rather than created inside a class. Purpose:** Reduce **tight coupling** between components.

How it works (PPT 例子): 通知系统本来在内部 `new EmailNotification()`，要加 SMS/WhatsApp 就得改 `NotificationSystem` 的 code。用 DI：`NotificationSystem` 不自己建服务，而是从**外部传进来**（通过 constructor），换通知方式不用改 `NotificationSystem`。结果：more flexible, easier to maintain, easier to test, less tightly coupled.

PPT Code:

```csharp
// Step 1: Interface
public interface INotification
{
    void Send();
}

// Step 2 & 3: Implementations
public class EmailNotification : INotification
{
    public void Send() { Console.WriteLine("Email Sent"); }
}
public class SMSNotification : INotification
{
    public void Send() { Console.WriteLine("SMS Sent"); }
}

// Step 4: Notification System (dependency injected via constructor)
public class NotificationSystem
{
    private INotification notification;
    public NotificationSystem(INotification notification)
    {
        this.notification = notification;   // injected from outside
    }
    public void NotifyUser()
    {
        notification.Send();
    }
}

// Step 5: Use Email  /  Step 6: Use SMS — just swap what you inject
INotification service = new EmailNotification();
NotificationSystem system = new NotificationSystem(service);
system.NotifyUser();
```

Q4b (10 marks) — 找出代码弱点 + 改进 ⭐⭐

这题会给你一段**"坏代码"**，让你找弱点 + 提改进。**好消息：PPT 里每个 pattern 都是为了解决一种弱点。**你只要认出代码犯了哪种毛病，就知道用哪个 pattern 改。下面是 5 种最可能考的弱点 + 改法（全部来自 PPT 描述的「问题」）：

代码弱点 (Weakness)	怎么认出	改进 (Improvement)
Tight coupling（紧耦合）	class 内部自己 `new` 出依赖（如 `new EmailNotification()`），加新功能要改这个 class	用 **Dependency Injection** — 从外部 constructor 传入依赖
Class explosion（类爆炸）	为每个功能组合建一个新类（`ReportWithCGPA`, `ReportWithAttendance`…）	用 **Decorator** — 一层层包装动态加功能
大量 if-else / switch 选算法	一堆 `if (type == "student") ... else if (type == "member") ...` 算折扣	用 **Strategy** — 每种算法封装成一个 strategy 类
接口不兼容（旧系统对接新系统）	旧 class 用 `ProcessPayment()`，新系统要 `MakePayment()`，硬改旧 code	用 **Adapter** — 建 adapter 桥接，不改旧 code
手动一个个通知对象	状态变了要手动逐个调用更新（没有自动通知机制）	用 **Observer** — Subject 变化时自动通知所有 Observers

📝 答题模板（10 分稳拿）

考试看到坏代码，按这个写：

1. **Identify the weakness** — 指出毛病（例：*"The `NotificationSystem` class creates the `EmailNotification` object directly inside itself. This causes **tight coupling** — whenever a new notification method is added, the `NotificationSystem` code must be modified."*）
2. **Why it's a problem** — 难维护 / 难测试 / 违反 Open-Closed principle（不该改已有 code 来加功能）。
3. **Improvement** — 用对应的 pattern 改（例：*"Apply **Dependency Injection**: pass the notification service through the constructor from outside, so the system can switch between Email, SMS, or WhatsApp without changing the `NotificationSystem` code."*）
4. **(可选) 给改进后的 code**（照上面 Q4a 对应 pattern 的 code 抄）。

> 💡 最常考的弱点八成是 **tight coupling → DI**，或 **class explosion → Decorator**，因为 PPT 把这两个的「问题」描述得最清楚。

🎯 考前 5 分钟速记

Q1 — 4 种 requirement：

- Functional = 系统做什么(动作) | Non-functional = 做得多好(快/安全/易用)
- User = 用户懂的高层语言 | System = 给开发者的技术细节(database/table)
- 每种都：先定义 → 是什么 → 基于 scenario → scenario 里找得到
- Q1b 访谈：Closed（固定问题）+ Open（开放探讨），好处=了解整体，坏处=不懂业务术语

Q2 — 架构：

- Layered(3-tier)：Presentation / Application / Data 三层
- Client-Server：Client / Web Server / App Server / DB Server
- 每个都写「定义+组成+优点+缺点」
- MVC 画图：User→Controller(Request)，Controller↔Model(Get Data)，Controller↔View(Get Presentation)，Controller→User(Response)；Controller 永不碰 data logic

Q3 — 测试：

- Verification = 照需求做对没(review,没跑程序) | Validation = 用户满意没(跑程序给用户测)
- Unit(单独测组件) → Integration(两模块一起测) → System(整个系统测) → UAT(客户测决定收不收)
- Test case 10 行表头：ID/Module/Description/Preconditions/Steps/Data/Expected/Actual/Status/Remarks

Q4 — 设计模式：

- 5 个：Adapter, Decorator (Structural) | Observer, Strategy (Behavioral) | DI (Dependency)
- Adapter=接口不兼容桥接 | Decorator=动态加功能不改原类 | Observer=一对多自动通知 | Strategy=换算法不改调用方 | DI=依赖外部传入降耦合
- Q4b 找弱点：tight coupling→DI | class explosion→Decorator | 大量if-else→Strategy | 接口不兼容→Adapter | 手动通知→Observer

---

老大，今晚把这份过一两遍，重点背 4 种 requirement 的定义、MVC 图、test case 表头、和 5 个 pattern 的一句话定义。睡前再看一遍速记表就行。加油 💪